

Имитационное моделирование
Лабораторная работа №5. Аппарат сетей Петри

Исаева Зарина Исмайлбековна

Содержание

1. Цель работы
2. Задание
3. Теоретическое введение
4. Выполнение лабораторной работы
5. Проверка результатов
6. Выводы
7. Список литературы

Список иллюстраций

1. Окружение Julia и структура проекта
2. Модуль сети Петри и параметрический сценарий в VS Code
3. Выполненные Jupyter notebook
4. Эволюция маркировки классической сети
5. Эволюция маркировки сети с арбитром
6. Контрольные кадры анимации
7. Двухпанельное сравнение позиций Eat
8. Результаты параметрической серии
9. Автоматические тесты

1. Цель работы

Цель работы — построить классическую сеть Петри для задачи обедающих философов, обнаружить взаимную блокировку, реализовать предотвращающую её сеть с арбитром и подтвердить различие стохастическими, детерминированными и параметрическими экспериментами.

2. Задание

Согласно главе 5 практикума, стр. 165–189 [1], требуется:

1. создать воспроизводимый Julia-проект и модуль `src/DiningPhilosophers.jl`;
2. построить классическую сеть и сеть с арбитром для пяти философов;
3. выполнить стохастическое и детерминированное моделирование;
4. сохранить полные траектории `dining_classic.csv` и `dining_arbiter.csv`;
5. построить для каждой сети четыре панели Think, Hungry, Eat и Fork;
6. создать GIF-анимацию классической сети при $N=3$, $T=30$ и 5 кадрах/с;
7. отдельным сценарием прочитать сохранённые CSV и построить две панели Eat;
8. преобразовать четыре сценария в литературный стиль и получить чистый JL, выполненный IPYNB, QMD и HTML;
9. выполнить параметрическую серию и интегрировать все листинги в отчёт.

3. Теоретическое введение

3.1 Формальное представление сети Петри

Сеть Петри задаётся множествами позиций P и переходов T , входными и выходными дугами и начальной маркировкой M_0 . Срабатывание разрешённого перехода t_j изменяет маркировку по правилу

$$M_{k+1} = M_k + (O - I)_{\cdot j},$$

где I и O — матрицы кратностей входных и выходных дуг. Раздельное хранение этих матриц позволяет корректно проверять разрешённость перехода, тогда как одна матрица инцидентности этого не гарантирует. Основные свойства сетей — ограниченность, сохраняемость, достижимость и живость [2,3].

3.2 Обедающие философы и deadlock

Пять философов конкурируют за пять вилок. В классической схеме каждый сначала берёт левую вилку, а затем правую. Маркировка, в которой все пять удерживают левую вилку и ожидают правую, не разрешает ни одного перехода. Это состояние является взаимной блокировкой. Арбитр выдаёт только $N-1$ разрешений на начало захвата ресурсов и тем самым разрывает условие циклического ожидания [4].

3.3 Стохастическое и детерминированное моделирование

Для дискретной траектории используется прямой алгоритм Гиллеспи [5]. При интенсивностях $a_j(M)$ время до следующего события распределено экспоненциально с параметром $a_0 = \sum_j a_j$, а переход выбирается пропорционально a_j . Непрерывная mass-action аппроксимация интегрируется методом Рунге—Кутты четвёртого порядка:

$$\frac{du}{dt} = (O - I)a(u).$$

Обе формы используют одну структуру сети и один вектор интенсивностей.

3.4 Воспроизводимое литературное программирование

DrWatson фиксирует структуру каталогов и активный проект [6]. Literate.jl сохраняет единственный комментированный исходник сценария и получает из него чистый JL, notebook и Quarto-документ, следуя принципам литературного программирования [7]. Все стохастические эксперименты используют явные seed.

4. Выполнение лабораторной работы

4.1 Окружение и структура проекта

В проекте закреплены Julia 1.11, DataFrames, CSV, CairoMakie, DrWatson, Literate и IJulia. Вычисления, изображения и документы воспроизводятся целями Makefile.

```

[deps]
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
CairoMakie = "13f3f980-e62b-5c42-98c6-ff1f3baf88f0"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Random = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"

[compat]
CairoMakie = "0.15"
DataFrames = "1"
DrWatson = "2"
IJulia = "1"
Literate = "2"
julia = "1.11"

[preferences.Makie]
precompile_workload = false

[preferences.CairoMakie]
precompile_workload = false
rhktrlwmd@Mac %

```

Рисунок 1: Зависимости проекта Julia

```

/workspace/labs/lab05/project
julia version 1.11.9
scripts/dining_philosophers.jl
scripts/dining_philosophers/dining_philosophers.jl
scripts/dining_philosophers__param.jl
scripts/dining_philosophers__param/dining_philosophers__param.jl
scripts/dining_philosophers_animation.jl
scripts/dining_philosophers_animation/dining_philosophers_animation.jl
scripts/dining_philosophers_report.jl
scripts/dining_philosophers_report/dining_philosophers_report.jl
scripts/setup_environment.jl
scripts/tangle.jl
src/DiningPhilosophers.jl
src/PetriDiningLab.jl
test/runtests.jl
rhktrlwmd@Mac %

```

Рисунок 2: Структура исходников и производных сценариев

```

name = "PetriDiningLab"
uuid = "64b9d50f-0898-4519-9d86-542c6fe5bb23"
authors = ["rhktrlwmd"]
version = "5.0.0"

[deps]
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
CairoMakie = "13f3f980-e62b-5c42-98c6-ff1f3baf88f0"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"

```

```
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4 299 306"  
Random = "9a3f8284-a2c9-5f02-9a11-845 980a1fd5c"  
Statistics = "10 745b16-79ce-11e8-11f9-7d13ad32a3b2"
```

[compat]

```
CairoMakie = "0.15"  
DataFrames = "1"  
DrWatson = "2"  
IJulia = "1"  
Literate = "2"  
julia = "1.11"
```

[preferences.Makie]

```
precompile_workload = false
```

[preferences.CairoMakie]

```
precompile_workload = false
```

Листинг 1 — полный файл `project/Project.toml`.

```
SOURCES = scripts/dining_philosophers.jl \  
          scripts/dining_philosophers_animation.jl \  
          scripts/dining_philosophers_report.jl \  
          scripts/dining_philosophers_param.jl
```

```
.PHONY: setup tangle run notebooks docs test verify all clean-results
```

setup:

```
julia --project=. scripts/setup_environment.jl
```

tangle:

```
julia --project=. scripts/tangle.jl $(SOURCES)
```

run:

```
@for source in $(SOURCES); do julia --project=. $$source || exit $$?. done
```

notebooks:

```
julia --project=. scripts/tangle.jl --execute $(SOURCES)
```

docs:

```
@for source in $(SOURCES); do \  
  name=$(basename $$source .jl); \  
  (cd notebooks/$$name && quarto render $$name.ipynb --to html \  
  --embed-resources --no-execute --output $$name.html) || exit $$?. \  
  mv notebooks/$$name/$$name.html markdown/$$name/$$name.html; \  
done
```

test:

```
julia --project=. test/runtests.jl
```

verify:

```
@clean=$(find scripts -mindepth 2 -maxdepth 2 -name '*.jl' | wc -l | tr -d ' '); \  
notebooks=$(find notebooks -mindepth 2 -maxdepth 2 -name '*.ipynb' | wc -l | tr -d ' '); \  
qmd=$(find markdown -mindepth 2 -maxdepth 2 -name '*.qmd' | wc -l | tr -d ' '); \  
html=$(find markdown -mindepth 2 -maxdepth 2 -name '*.html' | wc -l | tr -d ' '); \  
printf 'clean Julia scripts: %s\nexecuted notebooks: %s\nQuarto documents: %s\nHTML documents: %s\n' \  
  "$$clean" "$$notebooks" "$$qmd" "$$html"; \  
test "$$clean" = 4; test "$$notebooks" = 4; test "$$qmd" = 8; test "$$html" = 4; \  
test -s data/dining_classic.csv; test -s data/dining_arbiter.csv; \  
test -s plots/classic_simulation.png; test -s plots/arbiter_simulation.png; \  
test -s plots/philosophers_simulation.gif; test -s plots/final_report.png; \  
test -s plots/parameter_series.png
```

```
all: run notebooks docs test verify
```

clean-results:

```
rm -rf data plots notebooks markdown
```

Листинг 2 — полный файл `project/Makefile`.

using Pkg

```
project = normpath(joinpath(@_DIR_, "."))  
Pkg.activate(project)  
Pkg.instantiate()  
Pkg.precompile()
```

using IJulia

```
IJulia.installkernel("Julia lab05", "--project=$(project);"  
  specname="julia-lab05")
```

```
println("Активный проект: $(Base.active_project())")  
Pkg.status()
```

Листинг 3 — настройка среды и ядра Jupyter.

```
using DrWatson
@quickactivate "PetriDiningLab"
using Literate

execute_notebooks = "--execute" in ARGS
sources = filter(!("--execute"), ARGS)

function build_derivatives(source::AbstractString)
    name = splitext(basename(source))[1]
    script_target = scriptsdir(name)
    notebook_target = projectdir("notebooks", name)
    markdown_target = projectdir("markdown", name)
    foreach(mkpath, (script_target, notebook_target, markdown_target))

    Literate.script(source, script_target; name=name, credit=false)
    Literate.notebook(source, notebook_target; name=name,
        execute=execute_notebooks, credit=false)
    Literate.markdown(source, markdown_target; name=name,
        flavor=Literate.QuartoFlavor(), credit=false)
    Literate.markdown(source, markdown_target;
        name=name * "-report", flavor=Literate.QuartoFlavor(), credit=false,
        postprocess=text -> replace(
            replace(text, "{julia}" => "{julia}"),
            r"^(#{1,4}) "m => heading -> "##" * heading,
        ))
end

foreach(build_derivatives, sources)
```

Листинг 4 — генерация чистого кода, выполненных notebook и QMD.

4.2 Модуль сети Петри

Модуль сохраняет матрицы I и O отдельно. Результат стохастической симуляции включает полную траекторию, журнал переходов, признак deadlock и число завершённых приёмов пищи каждого философа.

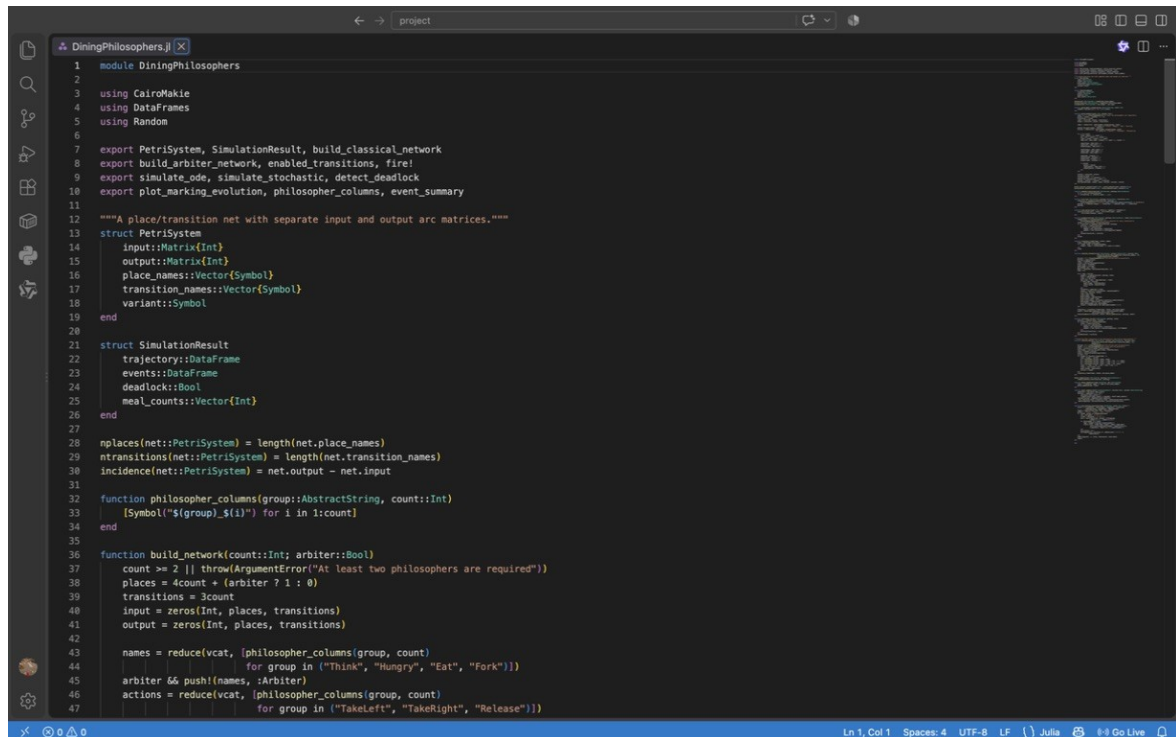


Рисунок 3: Общий модуль в настоящем VS Code

```
module PetriDiningLab
```



```
include("DiningPhilosophers.jl")
```

```
end
```

Листинг 5 — входной модуль Julia-проекта.

```
module DiningPhilosophers
```

```
using CairoMakie
using DataFrames
using Random
```

```
export PetriSystem, SimulationResult, build_classical_network
export build_arbiter_network, enabled_transitions, fire!
export simulate_ode, simulate_stochastic, detect_deadlock
export plot_marking_evolution, philosopher_columns, event_summary
```

```
"""A place/transition net with separate input and output arc matrices."""
```

```
struct PetriSystem
    input::Matrix{Int}
    output::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
    variant::Symbol
end
```

```
struct SimulationResult
    trajectory::DataFrame
    events::DataFrame
    deadlock::Bool
    meal_counts::Vector{Int}
end
```

```
nplaces(net::PetriSystem) = length(net.place_names)
ntransitions(net::PetriSystem) = length(net.transition_names)
incidence(net::PetriSystem) = net.output - net.input
```

```
function philosopher_columns(group::AbstractString, count::Int)
    [Symbol("$(group)_(i)") for i in 1:count]
end
```

```
function build_network(count::Int; arbiter::Bool)
    count >= 2 || throw(ArgumentError("At least two philosophers are required"))
    places = 4count + (arbiter ? 1 : 0)
    transitions = 3count
    input = zeros{Int, places, transitions}
    output = zeros{Int, places, transitions}
```

```
    names = reduce(vcat, [philosopher_columns(group, count)
                          for group in ("Think", "Hungry", "Eat", "Fork")])
    arbiter && push!(names, :Arbiter)
    actions = reduce(vcat, [philosopher_columns(group, count)
                           for group in ("TakeLeft", "TakeRight", "Release")])
```

```
    for i in 1:count
        think, hungry = i, count + i
        eat, left = 2count + i, 3count + i
        right = 3count + mod1(i + 1, count)
        take_left, take_right, release = i, count + i, 2count + i
```

```
        input[think, take_left] = 1
        input[left, take_left] = 1
        output[hungry, take_left] = 1
```

```
        input[hungry, take_right] = 1
        input[right, take_right] = 1
        output[eat, take_right] = 1
```

```
        input[eat, release] = 1
        output[think, release] = 1
        output[left, release] = 1
        output[right, release] = 1
```

```
    if arbiter
        permit = places
        input[permit, take_left] = 1
        output[permit, release] = 1
    end
```

```
end
```

```
initial = zeros{Int, places}
initial[1:count] .= 1
initial[(3count + 1):(4count)] .= 1
```

```

    arbiter && (initial[end] = count - 1)
    variant = arbiter ? :arbiter : :classical
    PetriSystem(input, output, names, actions, variant), initial
end

build_classical_network(count::Int) = build_network(count; arbiter=false)
build_arbiter_network(count::Int) = build_network(count; arbiter=true)

function enabled_transitions(net::PetriSystem, marking::AbstractVector)
    [j for j in 1:ntransitions(net)
     if all(marking .>= view(net.input, :, j))]
end

function fire!(net::PetriSystem, marking::Vector{Int}, transition::Int)
    transition in enabled_transitions(net, marking) ||
        throw(ArgumentError("Transition $(net.transition_names[transition]) is disabled"))
    marking .+= view(net.output, :, transition) .- view(net.input, :, transition)
    marking
end

function rate_vector(count::Int; left=2.4, right=0.8, release=1.1)
    vcat(fill(float(left), count), fill(float(right), count),
         fill(float(release), count))
end

function propensities(net::PetriSystem, marking::AbstractVector, rates::AbstractVector)
    length(rates) == ntransitions(net) ||
        throw(DimensionMismatch("One rate is required for every transition"))
    values = zeros{Float64, ntransitions(net)}
    for transition in enabled_transitions(net, marking)
        activity = rates[transition]
        for place in 1:nplaces(net)
            demand = net.input[place, transition]
            demand > 0 && (activity * = marking[place]^demand)
        end
        values[transition] = activity
    end
    values
end

function trajectory_frame(times, states, names)
    frame = DataFrame(time=times)
    for (index, name) in enumerate(names)
        frame[:, name] = [state[index] for state in states]
    end
    frame
end

function simulate_stochastic(net::PetriSystem, initial::Vector{Int}, horizon::Real;
    rates=rate_vector(div(length(net.transition_names), 3)),
    rng=MersenneTwister(202605))
    horizon > 0 || throw(ArgumentError("The horizon must be positive"))
    marking = copy(initial)
    times = Float64{0.0}
    states = Vector{Int}[copy(marking)]
    event_times = Float64[]
    event_names = String[]
    philosopher_ids = Int[]
    meals = zeros{Int, div(ntransitions(net), 3)}
    time = 0.0

    while time < horizon
        weights = propensities(net, marking, rates)
        total = sum(weights)
        total == 0 && break
        next_time = time - log(rand(rng)) / total
        if next_time > horizon
            push!(times, float(horizon))
            push!(states, copy(marking))
            break
        end
        threshold = rand(rng) * total
        chosen = findfirst(>=(threshold), cumsum(weights))
        fire!(net, marking, chosen)
        time = next_time
        push!(times, time)
        push!(states, copy(marking))
        push!(event_times, time)
        push!(event_names, string(net.transition_names[chosen]))
        philosopher = mod1(chosen, length(meals))
        push!(philosopher_ids, philosopher)
        chosen > 2*length(meals) && (meals[philosopher] += 1)
    end

    trajectory = trajectory_frame(times, states, net.place_names)

```

```

    events = DataFrame(time=event_times, transition=event_names,
                      philosopher=philosopher_ids)
    SimulationResult(trajecory, events, detect_deadlock(net, marking), meals)
end

function continuous_rhs(net::PetriSystem, marking, rates)
    activity = similar(rates, Float64)
    for transition in 1:ntransitions(net)
        value = rates[transition]
        for place in 1:nplaces(net)
            demand = net.input[place, transition]
            demand > 0 && (value *= max(marking[place], 0.0)^demand)
        end
        activity[transition] = value
    end
    incidence(net) * activity
end

"""Fixed-step RK4 integration of the deterministic mass-action approximation."""
function simulate_ode(net::PetriSystem, initial::AbstractVector, horizon::Real;
                    rates=rate_vector(div(length(net.transition_names), 3)),
                    saveat=0.1)
    horizon > 0 || throw(ArgumentError("The horizon must be positive"))
    saveat > 0 || throw(ArgumentError("saveat must be positive"))
    times = collect(0.0:saveat:float(horizon))
    last(times) < horizon && push!(times, float(horizon))
    state = Float64.(initial)
    states = Vector{Float64}[copy(state)]
    current = 0.0
    for target in Iterators.drop(times, 1)
        step = target - current
        k1 = continuous_rhs(net, state, rates)
        k2 = continuous_rhs(net, state .+ step .* k1 ./ 2, rates)
        k3 = continuous_rhs(net, state .+ step .* k2 ./ 2, rates)
        k4 = continuous_rhs(net, state .+ step .* k3, rates)
        state .+= step .* (k1 .+ 2k2 .+ 2k3 .+ k4) ./ 6
        state .+= max.(state, 0.0)
        push!(states, copy(state))
        current = target
    end
    trajectory_frame(times, states, net.place_names)
end

detect_deadlock(net::PetriSystem, marking::AbstractVector) =
    isempty(enabled_transitions(net, marking))

function detect_deadlock(frame::DataFrame, net::PetriSystem)
    final = [frame[end, name] for name in net.place_names]
    detect_deadlock(net, final)
end

function event_summary(result::SimulationResult, horizon::Real, variant::AbstractString)
    completed = sum(result.meal_counts)
    fairness = completed == 0 ? 0.0 :
        sum(result.meal_counts)^2 /
        (length(result.meal_counts) * sum(abs2, result.meal_counts))
    (variant=variant, deadlock=result.deadlock,
     last_time=last(result.trajectory.time), events=nrow(result.events),
     meals=completed, fairness=fairness, horizon=float(horizon))
end

function plot_marking_evolution(frame::DataFrame, count::Int; title="")
    colors = [:royalblue3, :darkorange2, :seagreen3, :firebrick2,
             :mediumpurple3, :goldenrod2, :deepskyblue3]
    figure = Figure(size=(1200, 900), fontsize=16)
    groups = ("Think", "Hungry", "Eat", "Fork")
    for (panel, group) in enumerate(groups)
        row = div(panel - 1, 2) + 1
        column = mod(panel - 1, 2) + 1
        axis = Axis(figure[row, column], title=group,
                   xlabel="Время", ylabel="Фишки")
        for philosopher in 1:count
            name = Symbol("$(group)_$(philosopher)")
            stairs!(axis, frame.time, frame[:, name]; step=:post,
                   color=colors[mod1(philosopher, length(colors))],
                   linewidth=2, label="Философ $(philosopher)")
        end
        axis.yticks = 0:1
        axislegend(axis; position=:rt, nbanks=count > 4 ? 2 : 1,
                  labels=11)
    end
    Label(figure[0, :], title, fontsize=23, font=:bold)
    figure
end

```

```
end
```

Листинг 6 — полный `project/src/DiningPhilosophers.jl`.

4.3 Базовый эксперимент

4.3.1 Классическая сеть и сеть с арбитром

Базовый опыт сопоставляет две сети Петри для пяти философов. Для каждой сети вычисляется дискретная траектория алгоритмом Гиллеспи и непрерывная mass-action аппроксимация. Все исходные маркировки сохраняются в CSV.

```
using DrWatson
@quickactivate "PetriDiningLab"

ENV["GKSwstype"] = "100"
using CairoMakie
using CSV
using DataFrames
using Random

include(srcdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
```

4.3.1.1 Параметры обязательного опыта

```
const PHILOSOPHERS = 5
const HORIZON = 50.0
const BASE_RATES = vcat(fill(2.4, PHILOSOPHERS),
                        fill(0.8, PHILOSOPHERS),
                        fill(1.1, PHILOSOPHERS))

mkpath(datadir())
mkpath(plotsdir())
```

4.3.1.2 Стохастические траектории

```
classic_net, classic_initial = build_classical_network(PHILOSOPHERS)
arbiter_net, arbiter_initial = build_arbiter_network(PHILOSOPHERS)

classic = simulate_stochastic(classic_net, classic_initial, HORIZON;
                             rates=BASE_RATES, rng=MersenneTwister(20260501))
arbiter = simulate_stochastic(arbiter_net, arbiter_initial, HORIZON;
                              rates=BASE_RATES, rng=MersenneTwister(20260502))

CSV.write(datadir("dining_classic.csv"), classic.trajectory)
CSV.write(datadir("dining_arbiter.csv"), arbiter.trajectory)
CSV.write(datadir("dining_classic_events.csv"), classic.events)
CSV.write(datadir("dining_arbiter_events.csv"), arbiter.events)
```

4.3.1.3 Детерминированная аппроксимация

```
classic_ode = simulate_ode(classic_net, classic_initial, HORIZON;
                           rates=BASE_RATES, saveat=0.1)
arbiter_ode = simulate_ode(arbiter_net, arbiter_initial, HORIZON;
                            rates=BASE_RATES, saveat=0.1)
CSV.write(datadir("dining_classic_ode.csv"), classic_ode)
CSV.write(datadir("dining_arbiter_ode.csv"), arbiter_ode)
```

4.3.1.4 Диагностика взаимной блокировки

```
summary = DataFrame([
    event_summary(classic, HORIZON, "Классическая сеть"),
    event_summary(arbiter, HORIZON, "Сеть с арбитром"),
])
CSV.write(datadir("dining_summary.csv"), summary)

println("=== Стохастические сети Петри: N=$(PHILOSOPHERS), T=$(HORIZON) ===")
show(summary; allrows=true, allcols=true)
```

```
println()
println("Deadlock обнаружен (классическая сеть): $(classic.deadlock)")
println("Deadlock обнаружен (сеть с арбитром): $(arbiter.deadlock)")
```

4.3.1.5 Четыре группы позиций классической сети

```
classic_figure = plot_marking_evolution(classic.trajectory, PHILOSOPHERS;
    title="Классическая сеть: развитие маркировки")
save(plotsdir("classic_simulation.png"), classic_figure, px_per_unit=1.5)
classic_figure
```

4.3.1.6 Четыре группы позиций сети с арбитром

```
arbiter_figure = plot_marking_evolution(arbiter.trajectory, PHILOSOPHERS;
    title="Сеть с арбитром: развитие маркировки")
save(plotsdir("arbiter_simulation.png"), arbiter_figure, px_per_unit=1.5)
arbiter_figure
```

Классическая модель останавливается в маркировке без разрешённых переходов. Ограничение арбитра оставляет один свободный ресурсный маршрут, поэтому модифицированная сеть продолжает работу до заданного горизонта.

Листинг 7 — полный литературный сценарий базового опыта.

При $N=5$ и $T=50$ классическая сеть выполнила пять захватов левой вилки и остановилась в момент 0,446562; `detect_deadlock` вернул `true`. Сеть с арбитром дошла до $T=50$, выполнила 76 переходов и 24 завершённых приёма пищи; `detect_deadlock` вернул `false`. Индекс равномерности Джеяна равен 0,859701.

Классическая сеть: развитие маркировки

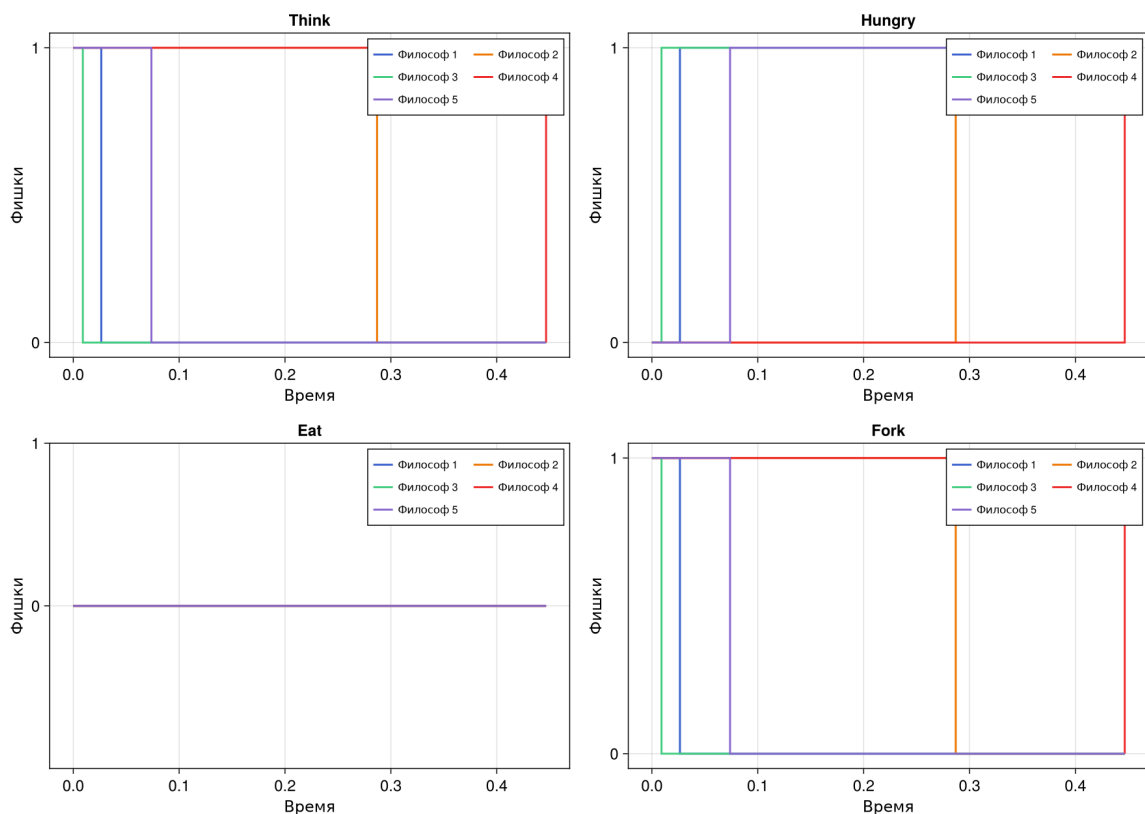


Рисунок 4: Классическая сеть: четыре обязательные панели

На Рисунок 4 линии Eat остаются нулевыми: все философы успели захватить по одной вилке и ни один не смог получить вторую.

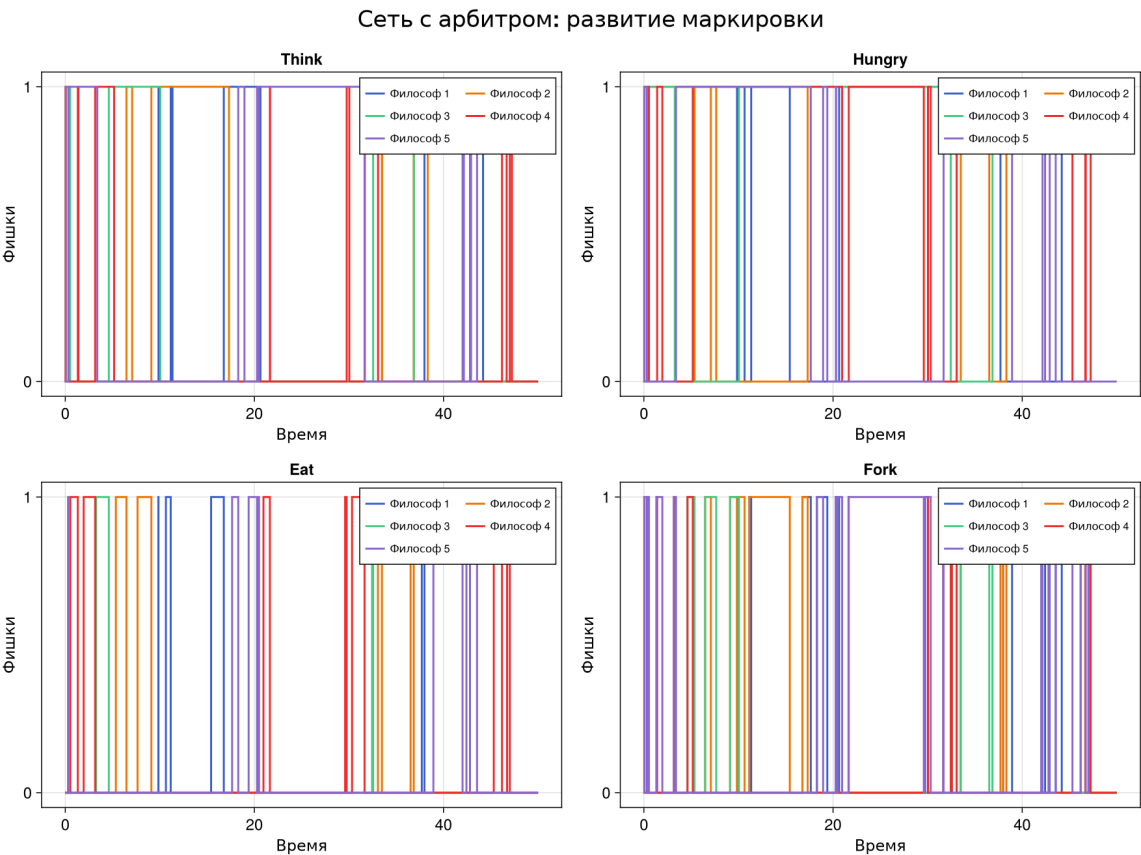


Рисунок 5: Сеть с арбитром: четыре обязательные панели

На Рисунок 5 чередование Think, Hungry и Eat продолжается до горизонта, а число свободных вилок изменяется согласованно с активными переходами.

4.4 Выполненный базовый notebook

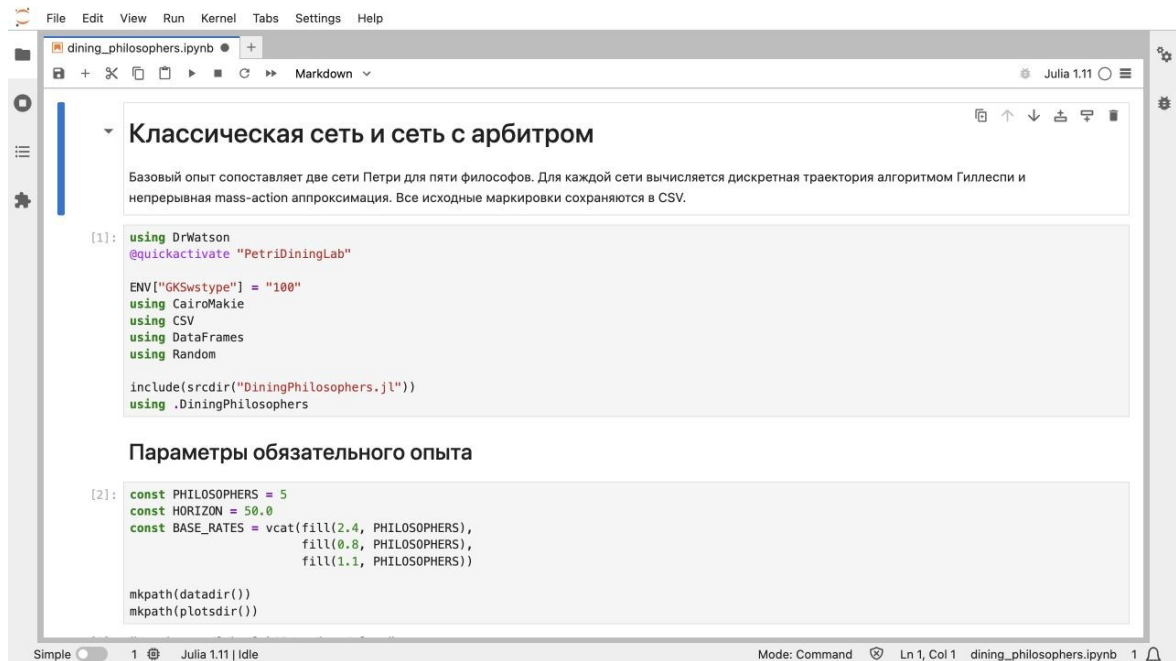


Рисунок 6: Параметры в выполненном Jupyter notebook

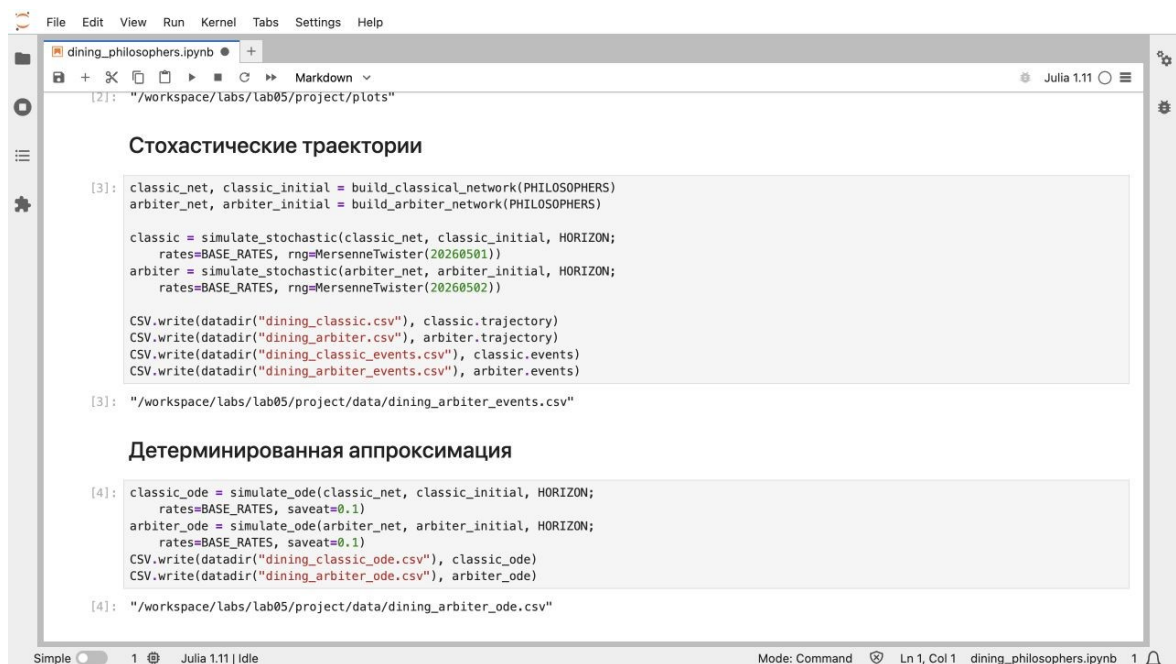


Рисунок 7: Табличный вывод и диагностика deadlock

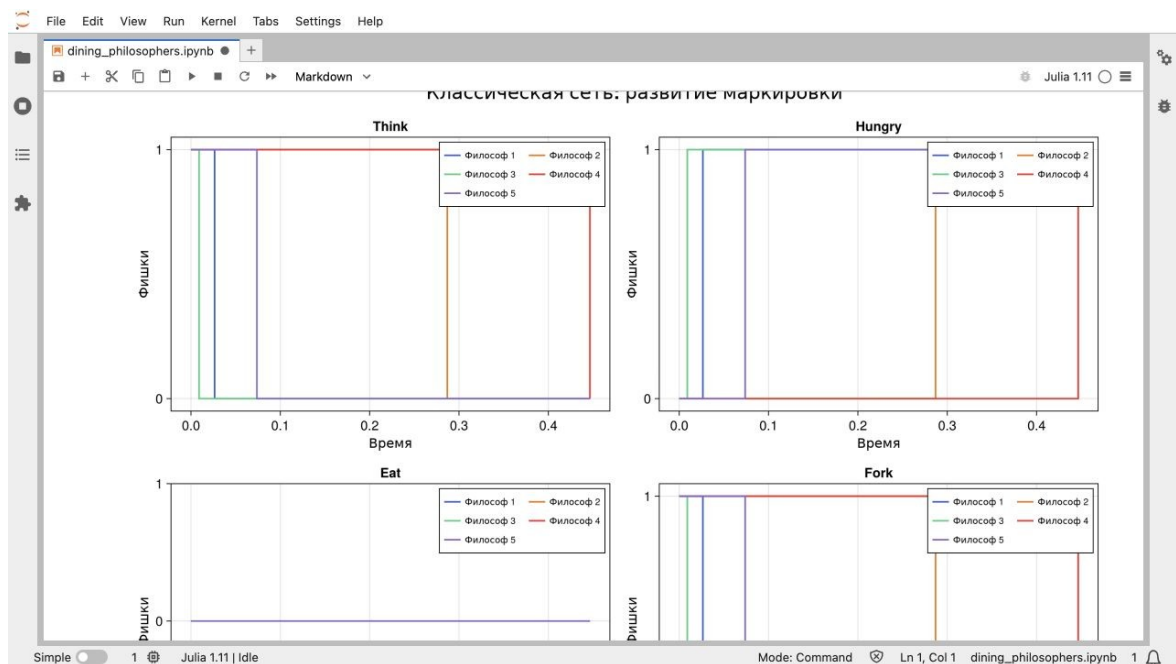


Рисунок 8: Встроенный четырёхпанельный график

Номера [1]–[6] и встроенный график подтверждают фактическое выполнение ячеек, а не только генерацию файла IPYNB.

4.5 Анимация маркировки

4.5.1 Анимация маркировки классической сети

Отдельный сценарий воспроизводит обязательный опыт для трёх философов на горизонте 30. Каждый кадр соответствует фактически сохранённой маркировке.

```
using DrWatson
@quickactivate "PetriDiningLab"

ENV["GKSwstype"] = "100"
using CairoMakie
using CSV
using DataFrames
using Random

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
```

4.5.1.1 Условия опыта

```
const PHILOSOPHERS = 3
const HORIZON = 30.0
const FRAME_RATE = 5
const RATES = vcat(fill(2.4, PHILOSOPHERS),
    fill(0.8, PHILOSOPHERS),
    fill(1.1, PHILOSOPHERS))

net, initial = build_classical_network(PHILOSOPHERS)
simulation = simulate_stochastic(net, initial, HORIZON;
    rates=RATES, rng=MersenneTwister(123))

mkpath(datadir("dining-animation"))
mkpath(plotsdir())
CSV.write(datadir("dining-animation", "trajectory.csv"),
    simulation.trajectory)
CSV.write(datadir("dining-animation", "events.csv"), simulation.events)
```


4.5.1.2 Построение кадров

```
trajectory = simulation.trajectory
labels = string(net.place_names)
frame = Observable(1)
values = @lift(Float64[trajectory[$frame, name] for name in net.place_names])
caption = @lift("Классическая сеть, t = " *
    string(round(trajectory.time[$frame]; digits=3)))

figure = Figure(size=(1200, 720), fontsize=16)
axis = Axis(figure[1, 1], title=caption, xlabel="Позиции сети Петри",
    ylabel="Число фишек", xticks=(1:length(labels), labels),
    xticklabelrotation=pi / 4, limits=(nothing, (0, 2.0)))
barplot!(axis, 1:length(labels), values;
    color=repeat([:royalblue3, :darkorange2, :seagreen3, :goldenrod2],
        inner=PHILOSOPHERS))

gif_path = plotsdir("philosophers_simulation.gif")
record(figure, gif_path, 1:nrow(trajectory); framerate=FRAME_RATE) do index
    frame[] = index
end
```

4.5.1.3 Контрольные маркировки

```
selected = unique(round.(Int,
    range(1, nrow(trajectory); length=min(4, nrow(trajectory)))))
keyframes = Figure(size=(1200, 760), fontsize=15)
for (panel, index) in enumerate(selected)
    row = div(panel - 1, 2) + 1
    column = mod(panel - 1, 2) + 1
    ax = Axis(keyframes[row, column],
        title="t=$(round(trajectory.time[index]; digits=3))",
        xticks=(1:length(labels), labels), xticklabelrotation=pi / 3,
        limits=(nothing, (0, 2.0)))
    barplot!(ax, 1:length(labels),
        Float64[trajectory[index, name] for name in net.place_names];
        color=:royalblue3)
end
save(plotsdir("philosophers_keyframes.png"), keyframes, px_per_unit=1.5)

animation_summary = DataFrame(
    philosophers=[PHILOSOPHERS], horizon=[HORIZON], fps=[FRAME_RATE],
    frames=[nrow(trajectory)], events=[nrow(simulation.events)],
    deadlock=[simulation.deadlock], final_time=[last(trajectory.time)],
)
CSV.write(datadir("dining-animation", "summary.csv"), animation_summary)
show(animation_summary; allrows=true, allcols=true)
println()
println("Анимация сохранена: $(relpath(gif_path, projectdir()))")
keyframes
```

Листинг 8 — полный сценарий анимации N=3, T=30, fps=5.

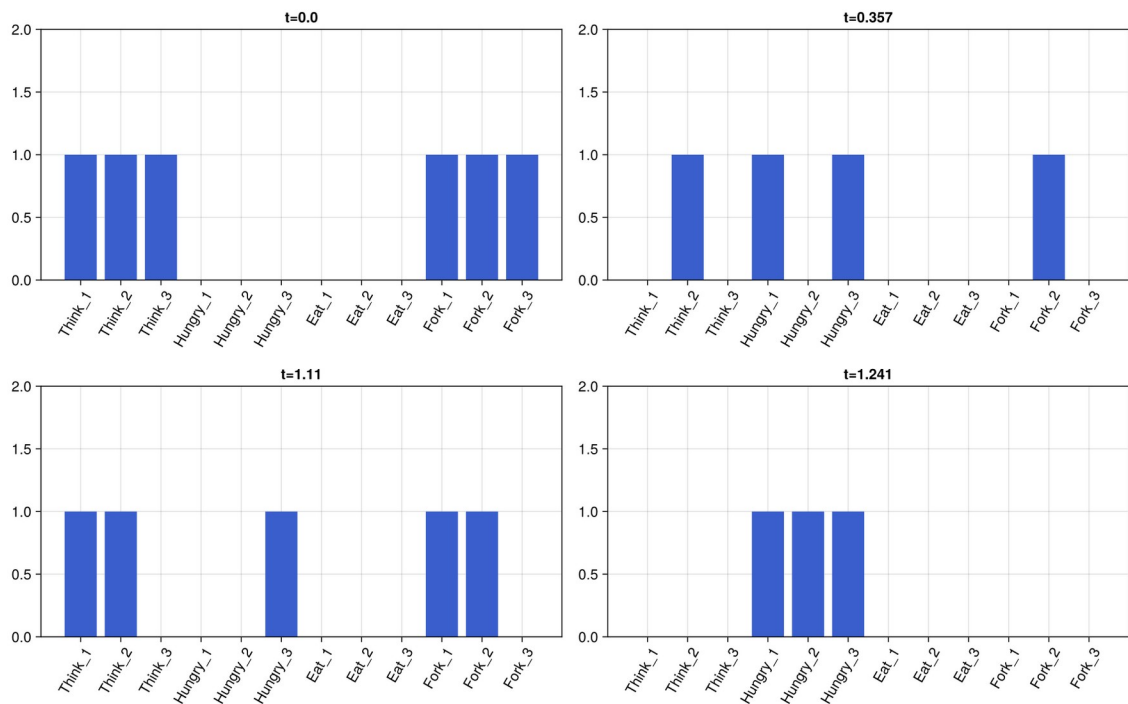


Рисунок 9: Начальная, промежуточные и конечная маркировки анимации

Для seed 123 получено шесть срабатываний и семь фактических маркировок. Deadlock возник в момент 1,240570. GIF формируется с требуемой частотой пять кадров в секунду из тех же строк траектории, которые сохранены в CSV.

4.6 Анализ ранее сохранённых CSV

4.6.1 Сравнение ранее сохранённых траекторий

Этот сценарий не запускает имитацию. Он читает CSV базового опыта, проверяет их структуру и строит двухпанельное сравнение только позиций

Eat_i.

```
using DrWatson
@quickactivate "PetriDiningLab"
```

```
ENV["GKSwstype"] = "100"
using CairoMakie
using CSV
using DataFrames
using Statistics
```

```
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
```

4.6.1.1 Чтение результатов базового опыта

```
const PHILOSOPHERS = 5
classic_path = datadir("dining_classic.csv")
arbiter_path = datadir("dining_arbiter.csv")
for path in (classic_path, arbiter_path)
    isfile(path) || error("Сначала выполните dining_philosophers.jl: " * path)
end

classic = CSV.read(classic_path, DataFrame)
arbiter = CSV.read(arbiter_path, DataFrame)
eat_columns = philosopher_columns("Eat", PHILOSOPHERS)
```

```

for frame in (classic, arbiter)
  all(column -> column in propertynames(frame), eat_columns) ||
    error("В сохранённой траектории отсутствуют позиции Eat_i")
  issorted(frame.time) || error("Временные отметки CSV не упорядочены")
end

```

4.6.1.2 Двухпанельная диаграмма

```

colors = [:royalblue3, :darkorange2, :seagreen3, :firebrick2, :mediumpurple3]
figure = Figure(size=(1150, 820), fontsize=16)
for (row, (name, frame)) in enumerate((("Классическая сеть", classic),
  ("Сеть с арбитром", arbiter)))
  axis = Axis(figure[row, 1], title=name, xlabel="Время",
    ylabel="Eat (0/1)", yticks=0:1)
  for philosopher in 1:PHILOSOPHERS
    stairs!(axis, frame.time, frame[:, eat_columns[philosopher]];
      step=:post, linewidth=2, color=colors[philosopher],
      label="Философ $(philosopher)")
  end
  axislegend(axis; position=:rt, nbanks=PHILOSOPHERS, labelsz=11)
end
save(plotsdir("final_report.png"), figure, px_per_unit=1.5)

```

4.6.1.3 Численные показатели из CSV

```

function activity_metrics(name, frame)
  eat_matrix = Matrix(frame[:, eat_columns])
  active_rows = vec(sum(eat_matrix; dims=2))
  (variant=name, rows=nrow(frame), final_time=last(frame.time),
    max_simultaneous=maximum(active_rows),
    active_share=mean(active_rows .> 0),
    zero_tail=all(iszero, active_rows[max(1, end - 3):end]))
end

comparison = DataFrame([
  activity_metrics("Классическая сеть", classic),
  activity_metrics("Сеть с арбитром", arbiter),
])
CSV.write(datadir("dining_report_summary.csv"), comparison)
show(comparison; allrows=true, allcols=true)
println()
figure

```

У классической сети завершающий нулевой участок связан с отсутствием разрешённых переходов. Для сети с арбитром CSV заканчивается на горизонте, поэтому нулевая строка сама по себе не интерпретируется как deadlock.

Листинг 9 — сценарий чтения двух CSV без повторного запуска модели.

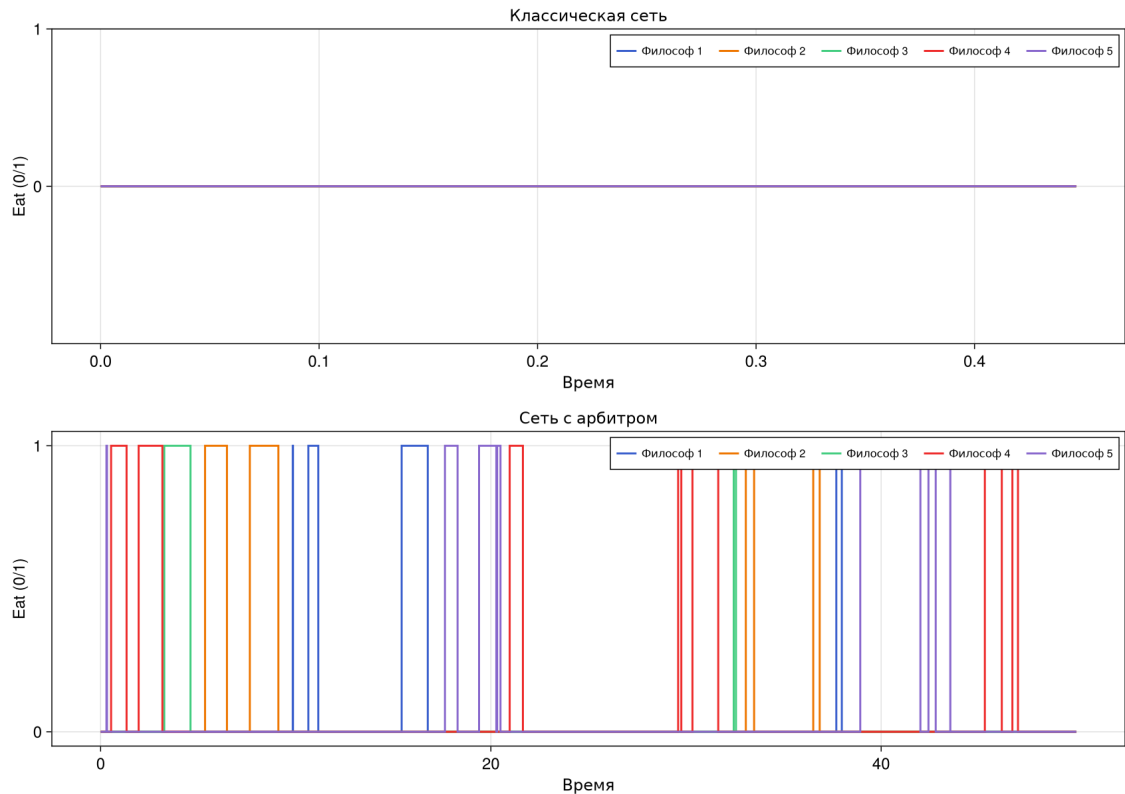


Рисунок 10: Сравнение Eat для классической сети и арбитра

Классический CSV содержит шесть маркировок и завершается нулевым Eat-хвостом. CSV арбитра содержит 78 строк, достигает $T=50$ и имеет активные Eat-состояния в 33,33% сохранённых маркировок. Максимум одновременно едящих равен единице, что согласуется с соседним расположением вилок.

4.7 Параметрическая серия

```

1  ## Параметрическая серия
2  #
3  # В серии изменяется число философов и интенсивность захвата второй вилки.
4  # Для каждой точки выполняются независимые повторы обеих структур сети.
5
6  using DrWatson
7  @quickactivate "PetriDiningLab"
8
9  ENV["GKSwttype"] = "100"
10 using CairoMakie
11 using CSV
12 using DataFrames
13 using Random
14 using Statistics
15
16 include(scrdir("DiningPhilosophers.jl"))
17 using .DiningPhilosophers
18
19 ## Сетка параметров
20
21 const COUNTS = (3, 5, 7)
22 const SECOND_FORK_RATES = (0.4, 0.8, 1.2)
23 const REPLICATIONS = 12
24 const HORIZON = 50.0
25
26 rows = NamedTuple{[]
27 for count in COUNTS, right_rate in SECOND_FORK_RATES,
28 replication in 1:REPLICATIONS
29 rates = vcat(fill(2.4, count), fill(right_rate, count), fill(1.1, count))
30 for (variant, builder, offset) in
31 (("Классическая", build_classical_network, 0),
32 ("С арбитражом", build_arbiter_network, 50_000))
33 net, initial = builder(count)
34 seed = 10_000count + 1_000round(Int, 10right_rate) +
35 replication + offset
36 result = simulate_stochastic(net, initial, HORIZON;
37 rates=rates, rng=MersenneTwister(seed))
38 meals = sum(result.meal_counts)
39 fairness = meals == 0 ? 0.0 :
40 meals^2 / (count * sum(abs2, result.meal_counts))
41 push!(rows, (philosophers=count, right_rate=right_rate,
42 replication=replication, variant=variant,
43 deadlock=result.deadlock, events=row(result.events),
44 meals=meals, throughput=meals / HORIZON,
45 fairness=fairness,
46 observed_until=last(result.trajectory.time)))
47 end

```

Рисунок 11: Параметрический сценарий в настоящем VS Code

4.7.1 Параметрическая серия

В серии изменяются число философов и интенсивность захвата второй вилки. Для каждой точки выполняются независимые повторы обеих структур сети.

```
using DrWatson
@quickactivate "PetriDiningLab"

ENV["GKSwstype"] = "100"
using CairoMakie
using CSV
using DataFrames
using Random
using Statistics

include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
```

4.7.1.1 Сетка параметров

```
const COUNTS = (3, 5, 7)
const SECOND_FORK_RATES = (0.4, 0.8, 1.2)
const REPLICATIONS = 12
const HORIZON = 50.0

rows = NamedTuple[]
for count in COUNTS, right_rate in SECOND_FORK_RATES,
  replication in 1:REPLICATIONS
  rates = vcat(fill(2.4, count), fill(right_rate, count), fill(1.1, count))
  for (variant, builder, offset) in
    (("Классическая", build_classical_network, 0),
     ("С арбитром", build_arbiter_network, 50_000))
    net, initial = builder(count)
    seed = 10_000count + 1_000round(Int, 10right_rate) +
      replication + offset
    result = simulate_stochastic(net, initial, HORIZON;
      rates=rates, rng=MersenneTwister(seed))
    meals = sum(result.meal_counts)
    fairness = meals == 0 ? 0.0 :
      meals^2 / (count * sum(abs2, result.meal_counts))
    push!(rows, (philosophers=count, right_rate=right_rate,
      replication=replication, variant=variant,
      deadlock=result.deadlock, events=nrow(result.events),
      meals=meals, throughput=meals / HORIZON,
      fairness=fairness,
      observed_until=last(result.trajectory.time)))
  end
end

runs = DataFrame(rows)
summary = combine(groupby(runs, [:philosophers, :right_rate, :variant]),
  :deadlock => mean => :deadlock_probability,
  :throughput => mean => :mean_throughput,
  :throughput => std => :sd_throughput,
  :fairness => mean => :mean_fairness,
  :events => mean => :mean_events)

mkpath(datadir("dining-parameter-series"))
CSV.write(datadir("dining-parameter-series", "replications.csv"), runs)
CSV.write(datadir("dining-parameter-series", "summary.csv"), summary)

println("=== Параметрическая серия сетей Петри ===")
println("Траекторий: $(nrow(runs))")
println("Агрегированных комбинаций: $(nrow(summary))")
show(summary; allrows=true, allcols=true)
println()
```

4.7.1.2 Вероятность deadlock и производительность

```
figure = Figure(size=(1200, 560), fontsize=16)
deadlock_axis = Axis(figure[1, 1], title="Вероятность deadlock",
```

```

        xlabel="Интенсивность захвата второй вилки",
        ylabel="Доля прогонов", limits=(nothing, (-0.04, 1.04)))
throughput_axis = Axis(figure[1, 2], title="Средняя производительность",
        xlabel="Интенсивность захвата второй вилки",
        ylabel="Приёмов пищи / ед. времени")

palette = Dict{3 => :royalblue3, 5 => :darkorange2, 7 => :seagreen3}
markers = Dict{"Классическая" => :circle, "С арбитром" => :rect}
for count in COUNTS, variant in ("Классическая", "С арбитром")
    subset = filter(row -> row.philosophers == count && row.variant == variant,
        summary)
    sort!(subset, :right_rate)
    label = "$(variant), N=$(count)"
    line_style = variant == "Классическая" ? :solid : :dash
    lines!(deadlock_axis, subset.right_rate, subset.deadlock_probability;
        color=palette[count], linestyle=line_style)
    scatter!(deadlock_axis, subset.right_rate, subset.deadlock_probability;
        color=palette[count], marker=markers[variant], label=label)
    lines!(throughput_axis, subset.right_rate, subset.mean_throughput;
        color=palette[count], linestyle=line_style)
    scatter!(throughput_axis, subset.right_rate, subset.mean_throughput;
        color=palette[count], marker=markers[variant], label=label)
end
axislegend(throughput_axis; position=:rt, labelsz=10)
save(plotsdir("parameter_series.png"), figure, px_per_unit=1.5)
figure

```

Повышение скорости получения второй вилки сокращает окно циклического ожидания. Арбитр устраняет глобальную блокировку во всех точках серии.

Листинг 10 — полный параметрический сценарий.

Серия содержит 216 траекторий: $N \in \{3, 5, 7\}$, интенсивность захвата второй вилки 0,4; 0,8; 1,2, двенадцать повторов и две структуры сети. Классическая сеть достигла deadlock во всех точках сетки. Для арбитра доля deadlock равна нулю, а средняя производительность при увеличении интенсивности возросла. Например, для $N=5$ она изменилась с 0,271667 до 0,563333 приёма пищи на единицу времени.

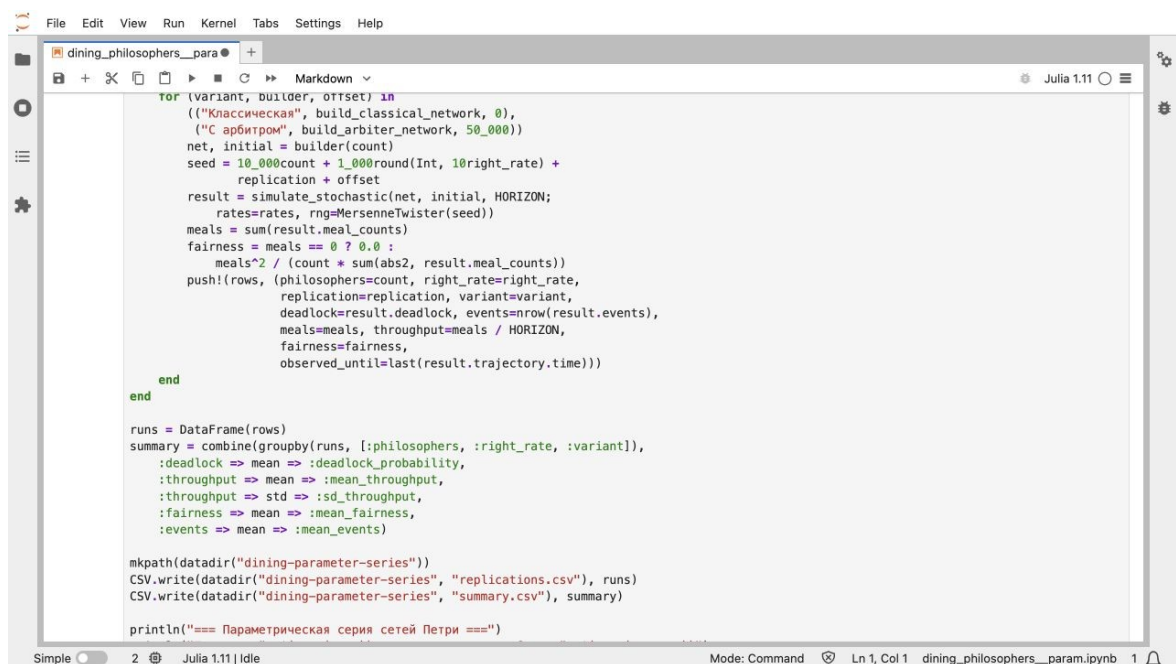


Рисунок 12: Выполненный параметрический notebook

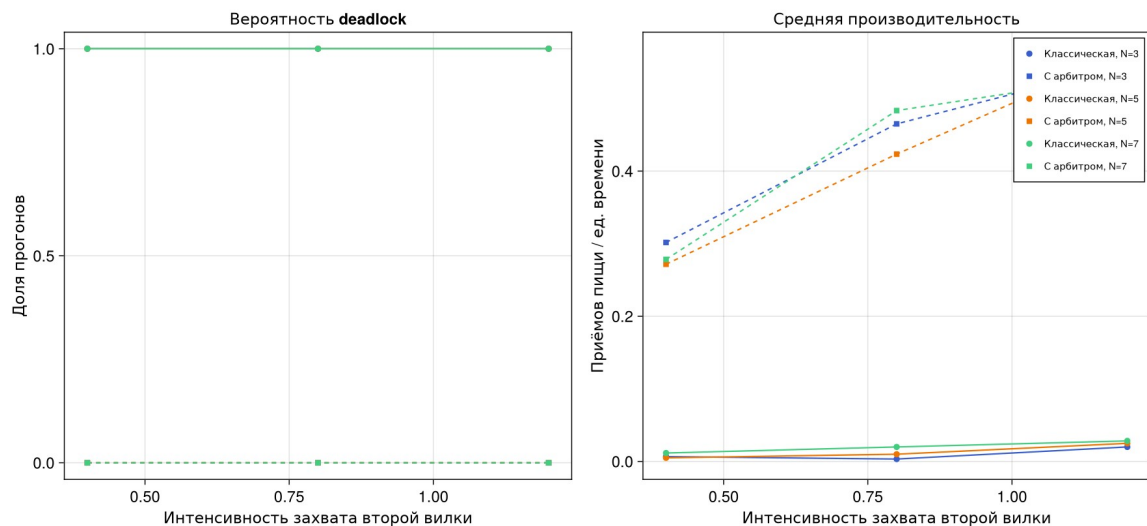


Рисунок 13: Вероятность блокировки и производительность

5. Проверка результатов

5.1 Автоматические проверки

Тесты проверяют размеры обеих сетей, начальные разрешённые переходы, неотрицательность маркировок, инвариант состояния каждого философа, известную тупиковую маркировку, обязательные параметры сохранённых CSV, сетку детерминированной аппроксимации и наличие всех производных форматов.

```
Test Summary: | Pass Total Time
Структура классической сети | 4 4 0.5s
Test Summary: | Pass Total Time
Арбитр и инварианты ресурсов | 9 9 0.1s
Test Summary: | Pass Total Time
Известная тупиковая маркировка | 1 1 0.0s
Test Summary: | Pass Total Time
Обязательные параметры и результаты | 4 4 1.7s
Test Summary: | Pass Total Time
Детерминированная аппроксимация | 3 3 0.4s
Test Summary: | Pass Total Time
Производные форматы всех сценариев | 20 20 0.0s
rhktrlwmd@Mac %
```

Рисунок 14: Результат полного набора тестов

```

using Test
using CSV
using DataFrames
using Random

include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers

@testset "Структура классической сети" begin
    net, initial = build_classical_network(5)
    @test size(net.input) == (20, 15)
    @test size(net.output) == (20, 15)
    @test sum(initial) == 10
    @test enabled_transitions(net, initial) == collect(1:5)
end

@testset "Арбитр и инварианты ресурсов" begin
    net, initial = build_arbiter_network(5)
    @test size(net.input) == (21, 15)
    @test initial[end] == 4
    result = simulate_stochastic(net, initial, 20.0;
        rng=MersenneTwister(605))
    @test !result.deadlock
    for philosopher in 1:5
        states = result.trajectory[!, Symbol("Think_$(philosopher)")] .+
            result.trajectory[!, Symbol("Hungry_$(philosopher)")] .+
            result.trajectory[!, Symbol("Eat_$(philosopher)")]
        @test all(states .== 1)
    end
    @test all(Matrix(result.trajectory[:, Not(:time)]) .>= 0)
end

@testset "Известная тупиковая маркировка" begin
    net, marking = build_classical_network(5)
    marking .= 0
    marking[6:10] .= 1
    @test detect_deadlock(net, marking)
end

@testset "Обязательные параметры и результаты" begin
    root = normpath(joinpath(@__DIR__, ".."))
    classic = CSV.read(joinpath(root, "data", "dining_classic.csv"), DataFrame)
    arbiter = CSV.read(joinpath(root, "data", "dining_arbiter.csv"), DataFrame)
    classic_net, _ = build_classical_network(5)
    arbiter_net, _ = build_arbiter_network(5)
    @test detect_deadlock(classic, classic_net)
    @test !detect_deadlock(arbiter, arbiter_net)
    @test last(arbiter.time) == 50.0
    @test all(Symbol("Eat_$(i)") in propertynames(classic) for i in 1:5)
end

@testset "Детерминированная аппроксимация" begin
    net, initial = build_arbiter_network(3)
    frame = simulate_ode(net, initial, 2.0; saveat=0.1)
    @test nrow(frame) == 21
    @test last(frame.time) == 2.0
    @test all(Matrix(frame[:, Not(:time)]) .>= 0)
end

@testset "Производные форматы всех сценариев" begin
    root = normpath(joinpath(@__DIR__, ".."))
    names = ("dining_philosophers", "dining_philosophers_animation",
        "dining_philosophers_report", "dining_philosophers__param")
    for name in names
        @test isfile(joinpath(root, "scripts", name, "$(name).jl"))
        @test isfile(joinpath(root, "notebooks", name, "$(name).ipynb"))
        @test isfile(joinpath(root, "markdown", name, "$(name).qmd"))
        notebook = read(joinpath(root, "notebooks", name, "$(name).ipynb"), String)
        @test occursin("execution_count":, notebook)
        @test !occursin("execution_count": null, notebook)
    end
end

```

Листинг 11 — полный `project/test/runtests.jl`.

Получена 41 успешная проверка в шести тестовых наборах. Дополнительно команда `make verify` подтвердила четыре чистых JL, четыре выполненных IPYNB, восемь QMD-фрагментов, четыре HTML-документа и все обязательные CSV, PNG и GIF.

5.2 Проверка сохраняемости

Для каждого философа во всех дискретных строках выполняется

$$Think_i + Hungry_i + Eat_i = 1.$$

Фишка философа не исчезает и не дублируется. Неотрицательность всех позиций проверяется по всей траектории арбитра. Deadlock определяется по множеству разрешённых переходов, а не по косвенному условию $Eat_i = 0$.

5.3 Противоречие в примере методички

Текст §5.2.11 требует 5 кадров/с, тогда как приведённая строка примера задаёт $fps=2$. В реализации выбран норматив из текстового требования: 5 кадров/с. Смысл опыта, $N=3$, $T=30$ и seed 123 сохранены.

6. Выводы

Построены две сети Петри для задачи обедающих философов. Классическая сеть воспроизводимо достигла глобальной блокировки, а ограничение $N-1$ разрешений арбитра исключило циклическое ожидание и обеспечило работу до $T=50$. Стохастический опыт подтверждён детерминированной аппроксимацией, полной историей событий, четырьмя группами маркировок и отдельным анализом CSV.

Параметрическая серия показала, что увеличение интенсивности захвата второй вилки повышает производительность сети с арбитром, но само по себе не устраняет структурную уязвимость классической схемы в выбранном диапазоне. Все четыре сценария воспроизведены из литературных источников в JL, IPYNB, QMD и HTML.

Список литературы

1. Королькова А. В., Кулябов Д. С. Имитационное моделирование: практикум. Москва: Российский университет дружбы народов имени Патриса Лумумбы, 2026.
2. Petri C. A. Kommunikation mit Automaten: Dissertation. Universität Hamburg, 1962.

3. Peterson J. L. [Petri Nets](#) // ACM Computing Surveys. 1977. Т. 9, № 3. С. 223–252.
4. Dijkstra E. W. Hierarchical Ordering of Sequential Processes: EWD310. Technological University Eindhoven, 1971.
5. Gillespie D. T. [Exact Stochastic Simulation of Coupled Chemical Reactions](#) // The Journal of Physical Chemistry. 1977. Т. 81, № 25. С. 2340–2361.
6. Datseris G. и др. [DrWatson: the perfect sidekick for your scientific inquiries](#) // Journal of Open Source Software. 2020. Т. 5, № 54. С. 2673.
7. Knuth D. E. [Literate Programming](#) // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.